Worksheet — 2.2 Problem Solving & Programming

OCR A Level Computer Science (H446) — Component 02, Section 2.2 (2.2.1 Programming techniques · 2.2.2 Computational methods). All code = OCR Exam Reference Language.

Name: ___ *Date:* ___

Time: ~35 min. Check against the answer key at the end.

1. Key-term match

Write the letter of the correct definition next to each term.

Term	Answer		Definition
1. Recursion	____	A	A subroutine that performs a task and returns a single value.
2. Base case	____	B	The region of a program where a variable can be accessed.
3. Scope	____	C	A subroutine that calls itself, with a stopping condition and a recursive case.
4. Function	____	D	Passing a copy of an argument so the original is unaffected.
5. Procedure	____	E	The condition that stops recursion — no further recursive call is made.
6. By value	____	F	Passing the memory address, so the original can be changed.
7. By reference	____	G	A stack of frames holding parameters, locals and return addresses for active calls.
8. Call stack	____	H	A subroutine that performs a task but returns no value.

2. Predict the output — variable SCOPE (local vs global)

Study the code, then answer below. Remember: a **local** variable declared inside a subroutine is separate from a **global** of the same name and **shadows** it inside that subroutine.

```
global score = 10

procedure addPoints()
    score = 99          // refers to the GLOBAL score
endprocedure

procedure tryLocal()
    score = 5           // LOCAL score, declared inside here
    print(score)
endprocedure

print(score)
tryLocal()
addPoints()
print(score)
```

a) What is printed by the **first** `print(score)`? _____

b) What is printed **inside** `tryLocal()`? _____

c) What is printed by the **last** `print(score)`? _____

d) Explain, in one sentence, why the assignment inside `tryLocal()` does **not** change what the final `print` shows.

3. Predict the output — pass BY VALUE vs BY REFERENCE

The same subroutine is called twice. Recall: **by value** passes a **copy** (original unchanged); **by reference** passes the **address** (original can be changed).

```
procedure increase(byVal n)
    n = n + 100
endprocedure

procedure boost(byRef n)
    n = n + 100
endprocedure

a = 1
increase(a)
print(a)

b = 1
boost(b)
print(b)
```

a) What is printed for `a`? _____

b) What is printed for `b`? _____

c) State the **consequence** (the marks-worthy point) that explains the difference — not just "a copy is made".

4. Recursion TRACE — complete the call stack

Here is a recursive factorial function:

```
function factorial(n)
    if n == 0 then
        return 1
    else
        return n * factorial(n - 1)
    endif
endfunction

print(factorial(4))
```

(a) Complete the table of stack frames as they are **pushed** (calls going down), and the **returned value** as each frame is **popped** (LIFO, going back up). One row is done for you.

Call (frame pushed)	Base case reached?	Expression returned	Value returned (on pop)
factorial(4)	No	4 * factorial(3)	_
factorial(3)	No	3 * factorial(2)	_
factorial(2)	No	_____	_
factorial(1)	No	_____	_
factorial(0)	**Yes**	return 1	**1**

(b) Final value printed by `print(factorial(4))`: _____

(c) What error occurs, and why, if the base case `if n == 0` were removed?

5. Write/complete a class in OCR pseudocode (with inheritance)

Complete the two classes below. `Vehicle` should store **make** and **numWheels** as **private** attributes set by a constructor, plus a method `describe()` that prints them. `Car` should **inherit** from `Vehicle`, add a private attribute **doors**, and **override** `describe()` to also print the number of doors.

```
class Vehicle
    private make
    private numWheels

    public procedure new(m, w)
        ____________ = m
        ____________ = w
    endprocedure

    public procedure describe()
        print("Make: " + make + ", wheels: " + numWheels)
    endprocedure
endclass

class Car _____________ Vehicle      // keyword for inheritance
    private doors

    public procedure new(m, w, d)
        super.new(___________, ___________)   // call the parent constructor
        doors = ___________
    endprocedure

    public procedure describe()
        // override: reuse parent then add doors
        ___________________________________
        print("Doors: " + doors)
    endprocedure
endclass

myCar = new Car("Ford", 4, 5)
myCar.describe()
```

a) Fill in every blank above.

b) Name the OOP principle shown by keeping `make` / `numWheels` **private** and only changing them through methods. __`_____`

c) Name the OOP principle shown when `Car`'s `describe()` replaces the inherited one. __`_____`

---

6. Match the computational method to its use

Methods: **Backtracking · Data mining · Heuristics · Performance modelling · Pipelining · Visualisation**

Write the best-fit method next to each use.

a) Splitting CPU instruction handling into fetch–decode–execute **stages** so different instructions are processed at once. → __`_____`

b) Searching a supermarket's huge loyalty-card records for hidden buying patterns and correlations. → __

c) A satnav using a **rule-of-thumb** (A*) to find a good-enough route quickly rather than the guaranteed-best one. → __

d) Solving a Sudoku by trying a value, and on a **dead end** returning to the last choice to try another. → __

e) Using a simulation to predict server load before the system is built. → __

f) Turning data into heat maps and charts to reveal trends and aid decisions. → __

7. Challenge box

Challenge — convert iteration to recursion.

Here is an **iterative** countdown procedure:

```
procedure countdown(n) while n > 0 print(n) n = n - 1 endwhile print("Liftoff") endprocedure
```

(i) Rewrite it as a **recursive** procedure `countdownR(n)` with the same output. Clearly mark your **base case** and your **recursive case**.

``` procedure countdownR(n)

_____

_____

_____

_____

endprocedure ```

(ii) State **one** advantage and **one** disadvantage of the recursive version compared with the iterative one.

## Answer key

**1. Key-term match:** 1 → C; 2 → E; 3 → B; 4 → A; 5 → H; 6 → D; 7 → F; 8 → G.

**2. Scope (worked):** - a) **10** — the global `score`, before any subroutine runs. - b) **5** — the local `score` inside `tryLocal()`. - c) **99** — `addPoints()` assigned to the **global** `score`, so the change persists. - d)
```

The `score` inside `tryLocal()` is a **local** variable that **shadows** the global only within that subroutine; it is destroyed when the subroutine ends, so it never affects the global value the final `print` reads.

3. By value vs by reference (worked): - a) `a` prints **1** — `increase` received a **copy** (by value), so the original was unchanged. - b) `b` prints **101** — `boost` received the **address** (by reference), so adding 100 changed the original. - c) The consequence: with **by value** the original variable is **unchanged** because only a copy is altered; with **by reference** the original **can be (and is) changed** because the subroutine works on the same memory location.

4. Recursion trace:

Call (frame pushed)	Base case?	Expression returned	Value returned (on pop)
factorial(4)	No	4 * factorial(3)	4 * 6 = **24**
factorial(3)	No	3 * factorial(2)	3 * 2 = **6**
factorial(2)	No	2 * factorial(1)	2 * 1 = **2**
factorial(1)	No	1 * factorial(0)	1 * 1 = **1**
factorial(0)	**Yes**	return 1	**1**

Frames are **pushed** down to `factorial(0)`, then **popped** back up in LIFO order, each multiplying its `n` by the value returned from below.

- (b) Final value printed: **24**.
- (c) Without the base case, `factorial(0)` would call `factorial(-1)`, then `factorial(-2)`, … with **no stopping condition**. Each call **pushes another stack frame**; the call stack fills up and a **stack overflow** error occurs.

5. Class (model answer):

```
class Vehicle
    private make
    private numWheels

    public procedure new(m, w)
        make = m
        numWheels = w
    endprocedure

    public procedure describe()
        print("Make: " + make + ", wheels: " + numWheels)
    endprocedure
endclass

class Car inherits Vehicle
    private doors

    public procedure new(m, w, d)
        super.new(m, w)
        doors = d
    endprocedure

    public procedure describe()
        super.describe()
        print("Doors: " + doors)
    endprocedure
endclass

myCar = new Car("Ford", 4, 5)
myCar.describe()
```

Output:

```
Make: Ford, wheels: 4
Doors: 5
```

- b) **Encapsulation** (data hiding — private attributes accessed only via methods).
- c) **Polymorphism** via **overriding** (the subclass replaces the inherited method with the same name and parameters).

6. Computational methods: a) **Pipelining**; b) **Data mining**; c) **Heuristics**; d) **Backtracking**; e) **Performance modelling**; f) **Visualisation**.

7. Challenge (model answer):

```
procedure countdownR(n)
    if n == 0 then              // BASE CASE
        print("Liftoff")
    else                        // RECURSIVE CASE
        print(n)
        countdownR(n - 1)
    endif
endprocedure
```

- (ii) **Advantage** (any one): shorter/more elegant code that mirrors the self-similar structure; no explicit loop variable to manage. **Disadvantage** (any one): uses more memory (a stack frame per call); slower due to call overhead; risk of **stack overflow** for large `n`.